

Advanced C Programming Lab

Session III – Mar 26

Objectives

We will be talking about;

- Input/Output Library
- “scanf()” Function

Input/Output Library

In C, there are multiple libraries for different use cases. For now, we will just be focusing on `<stdio.h>`. Some example functions are;

Function	Description
fopen()	Opens a file and returns a file pointer for use in file handling functions
fclose()	Closes a file
remove()	Deletes a file
rename()	Changes the name of a file
fgetc()	Returns the ASCII value of a character in a file and advances the position indicator
getc()	The same as fgetc()
getchar()	Reads one character of user input and returns its ASCII value
scanf()	Reads formatted data from user input and writes it into a number of memory locations
printf()	Writes a formatted string to the console

Input/Output Library

```
#include <stdio.h>
int main() {
  // Create an integer variable that will
  //store the number we get from the user
  int myNum;
  // Ask the user to type a number
  printf("Type a number and press enter: \n");
  // Get and save the number the user types
  scanf("%d", &myNum);
  // Print the number the user typed
  printf("Inputted number is: %d", myNum);
  return 0;
}
```

Output:

```
Type a number and press
enter: …
Inputted number is: …
```

Input/Output Library

```
#include <stdio.h>
int main() {
  // Create an integer variable that will
  //store the number we get from the user
  int myNum;
  // Ask the user to type a number
  printf("Type a number and press enter: \n");
  // Get and save the number the user types
  scanf("%d", &myNum);
  // Print the number the user typed
  printf("Inputted number is: %d", myNum);
  return 0;
}
```

An important note is that the `scanf()` function takes two arguments:

- The format specifier of our variable (%d).
- The reference operator, which stores the memory address of out variable (&myNum instead of myNum).

scanf()

There are many format specifiers recognized by scanf but we will be focusing on the most used ones for now in the list below:

type	Qualifying Input	Type of argument
c	Single character: Reads the next character. If a width different from 1 is specified, the function reads width characters and stores them in the successive locations of the array passed as argument. No null character is appended at the end.	char *
d	Decimal integer: Number optionally preceded with a + or - sign	int *
e, E, f, g, G	Floating point: Decimal number containing a decimal point, optionally preceded by a + or - sign and optionally followed by the e or E character and a decimal number. Two examples of valid entries are -732.103 and 7.12e4	float *
o	Octal Integer:	int *
s	String of characters. This will read subsequent characters until a whitespace is found (whitespace characters are considered to be blank, newline and tab).	char *
u	Unsigned decimal integer.	unsigned int *
x, X	Hexadecimal Integer	int *

scanf()

In C, scanf() is a standard input function used to read formatted data from the standard input stream (stdin), which is usually the keyboard.

```
#include <stdio.h>
int main() {
    int n;
    // Reading an integer input
    scanf("%d", &n);
    printf("%d", n);
    return 0;
}
```

Output:

(inputted number by user will be printed after pressing Enter)

scanf()

A program with single type input.

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    int result = scanf("%d", &num);
    if (result == 1) {
        printf("You entered: %d\n", num);
    } else {
        printf("Failed to read an integer.\n");
    }
    return 0;
}
```

Output:

```
Enter an integer: …
You entered: …
```

scanf()

A program with different types of input.

```
#include <stdio.h>
int main() {
   int age;
   float height;
   char initial;
   printf("Enter your age, height (in meters), and first initial: ");
   int result = scanf("%d %f %c", &age, &height, &initial);
   if (result == 3) {
       printf("You entered: Age = %d, Height = %.2f, Initial = %c\n", age, height, initial);
   } else {
       printf("Failed to read all values.\n");
   }
   return 0;
}
```

Output:

```
Enter your age, height (in meters), and first initial: 20 170 JB
You entered: Age = 20, Height = 170.00, Initial = JB
```

scanf()

Hexadecimal to Decimal number convertor.

```
#include <stdio.h>
int main() {
    int num;
scanf("%x", &num);
printf("%d", num);
}
```

Input:

```
0A
```

Output:

```
10
```

HEXADECIMAL VALUES																DECIMAL RANGE
00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	00-15
10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F	16-31
20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F	32-47
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F	48-63
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F	64-79
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F	80-95
60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F	96-111
70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F	112-127
80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	128-143
90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F	144-159
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF	160-175
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF	176-191
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	192-207
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	208-223
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF	224-239
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF	240-255

© TechTerms.com

scanf()

Using scanf() to print multiple words.

```
#include <stdio.h>
int main() {
    char name[40];
    // Reading an inputted string
    printf("Type your full name: \n");
    scanf("%s", name);
    printf("Hello! %s", name);
    return 0;
}
```

Input:

```
Johny Blaze
```

Output:

```
Hello! Johny
```

*When working with string in scanf(), make sure to specify an array size large enough to fit your data. Here we have used 40 to make sure it can handle long names.

scanf()

Fixing scanf() limitations.

The issue with scanf() is that it stops scanning for characters when it encounters whitespace, tabs, etc. So, to display more than a single word, we can use:

- A feature of scanf() called **scanset characters %[]** which allows us to read a sequence of characters until a certain condition is met.
- We can use another function in stdio.h library called **fgets()**, which is capable of **reading a line of text**. Don't forget to provide the name of string, sizeof(string_name), and stdin in the parentheses.

scanf()

Fixing scanf() limitations.

```c
#include <stdio.h>
int main() {
    char name[40];
    // Reading an inputted string
    printf("Type your full name: \n");
    scanf("%[^\n]", name);
    printf("Hello! %s", name);
    return 0;
}
```

Input:

```
Johny Blaze
```

```c
#include <stdio.h>
int main() {
    char name[40];
    // Reading an inputted string
    printf("Type your full name: \n");
    fgets(name, sizeof(name), stdin);
    printf("Hello! %s", name);
    return 0;
}
```

Output:

```
Hello! Johny Blaze
```

Thank you for your time and attention!

References:

- https://www.geeksforgeeks.org/
- https://www.w3schools.com/
- https://www.tutorialspoint.com/
- https://gemini.google.com/